

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет прикладної математики
Кафедра системного програмування і спеціалізованих
комп'ютерних систем**

РОЗРАХУНКОВО-ГРАФІЧНА РОБОТА

*з дисципліни
«Бази даних та засоби управління»*

**ТЕМА: “Створення додатку бази даних, орієнтованого на
взаємодію з СУБД PostgreSQL”**

**Група: КВ-34
Виконав: Протченко П.О
Перевірив:
Оцінка:**

Вступ.....	4
1.Порядок виконання роботи.....	5
2.Виконання.....	6
2.1. Структура бази даних з лабораторної роботи №1	6
2.1.1. Реляційна схема БД.	6
2.1.2. Зв'язки	7
2.1.3. Графічне подання концептуальної моделі «Сутність-зв'язок» зображено на рисунку 1 та рисунку 2 у PostgreSQL	8
2.2. Схема меню користувача та використані технології.....	10
2.2.1. Схема меню користувача	10
2.2.2. Мова програмування та використані бібліотеки.....	12
Перше деталізоване завдання	13
2.3. Вилучення запису батьківської таблиці та поведінка дочірньої (контроль 1:N)	13
2.3.1. Випадок А — видалення можливе (немає залежних даних)..	14
2.3.2. Випадок Б — видалення неможливе (є залежні дані).....	17
2.4. Вставка в дочірню таблицю та поведінка за відсутнього батьківського запису	19
2.4.1. Дочірня таблиця Enrollment — перевірка існування батьків (Student/Course/Instructor).....	20
2.4.2. Дочірня таблиця Exam — спроба вставки без існуючої трійки в Enrollment (демонстрація перехоплення помилки СУБД) ...	22
Друге деталізоване завдання	24
2.5. Копії SQL-запитів, що ілюструють генерацію при визначених вхідних параметрах	24
2.6. Копії екрана (ілюстрації) з фрагментами згенерованих даних таблиць(а також фото з pgAdmin).....	28
Третє деталізоване завдання.....	31
2.7. Ілюстрації введення пошукового запиту та результатів виконання.....	31
2.8. Копії SQL-запитів із зазначеними початковими параметрами .	33
Четверте деталізоване завдання	36

2.9. Ілюстрації коду модуля Model (MVC) та короткий опис функцій	36
3. Висновок:	50

Вступ

Мета: здобуття вмінь програмування прикладних додатків баз даних PostgreSQL.

Варіант

Система обліку екзаменаційних балів студентів

Посилання на GitHub

[GitHub](#)

Телеграм

@PR0TX

1.Порядок виконання роботи

1. Реалізувати функції перегляду, внесення, редагування та вилучення даних у таблицях бази даних, створених у лабораторній роботі №1, засобами консольного інтерфейсу.
2. Передбачити автоматичне пакетне генерування «рандомізованих» даних у базі.
3. Забезпечити реалізацію пошуку за декількома атрибутами з двох та більше сутностей одночасно: для числових атрибутів – у рамках діапазону, для рядкових – як шаблон функції LIKE оператора SELECT SQL, для логічного типу – значення True/False, для дат – у рамках діапазону дат.
4. Програмний код виконати згідно шаблону MVC (модель-подання-контролер).

2.Виконання

2.1. Структура бази даних з лабораторної роботи №1

2.1.1. Реляційна схема БД.

[PK] — первинний ключ

[FK] — зовнішній ключ

[UQ] — унікальність

[NN] — NOT NULL

[CHK] — перевірка діапазону/правила

IDENTITY — автоінкремент

Таблиця 1 – Опис об'єктів бази даних

Сутність	Атрибут	Тип атрибуту / обмеження
Instructor	instructor_id	integer [PK, IDENTITY]
	last_name	varchar(60) [NN]
	first_name	varchar(60) [NN]
	email	varchar(120) [NN, UQ]
	department	varchar(80) [NN]
Student	student_id	integer [PK, IDENTITY]
	last_name	varchar(60) [NN]
	first_name	varchar(60) [NN]
	email	varchar(120) [NN, UQ]
	group_code	varchar(20) [NN]
Course	course_id	integer [PK, IDENTITY]
	title	varchar(120) [NN]
	credits	smallint [NN]
	semester	smallint [NN]
Enrollment (тріпка)	course_id	integer [PK(1), FK → Course(course_id); ON UPDATE CASCADE; ON DELETE CASCADE]
	student_id	integer [PK(2), FK → Student(student_id); ON UPDATE CASCADE; ON DELETE CASCADE]
	instructor_id	integer [PK(3), FK → Instructor(instructor_id); ON UPDATE CASCADE; ON DELETE CASCADE]
Exam	student_id	integer [PK(1), FK(пазом із course_id, instructor_id) → Enrollment(course_id, student_id, instructor_id); ON UPDATE CASCADE; ON DELETE CASCADE]
	course_id	integer [PK(2), FK(пазом із student_id, instructor_id) → Enrollment(course_id, student_id, instructor_id)]
	instructor_id	integer [PK(3), FK(пазом із student_id, course_id) → Enrollment(course_id, student_id, instructor_id)]
	attempt_no	smallint [PK(4), NN]
	document	varchar(255) [NN]
	exam_date	date [NN]
	grade	smallint [NN]

2.1.2. Зв'язки

Student – Course – Instructor (3-мірний зв'язок) реалізовано асоціативною сутністю **Enrollment**:

PK Enrollment = $\langle \text{course_id}, \text{student_id}, \text{instructor_id} \rangle$.

Зовнішні ключі:

- Enrollment.course_id FK $\rightarrow$ Course(course_id) (*ON UPDATE CASCADE, ON DELETE CASCADE*);
- Enrollment.student_id FK $\rightarrow$ Student(student_id) (*ON UPDATE CASCADE, ON DELETE CASCADE*);
- Enrollment.instructor_id FK $\rightarrow$ Instructor(instructor_id) (*ON UPDATE CASCADE, ON DELETE CASCADE*).

Enrollment – Exam : 1:N (тріадне призначення має багато іспитів/спроб):

PK Exam = $\langle \text{student_id}, \text{course_id}, \text{instructor_id}, \text{attempt_no} \rangle$.

Складений FK: Exam(student_id, course_id, instructor_id) до Enrollment(course_id, student_id, instructor_id) (*ON UPDATE CASCADE, ON DELETE CASCADE*).

2.1.3. Графічне подання концептуальної моделі «Сутність-зв'язок» зображено на рисунку 1 та рисунку 2 у PostgreSQL

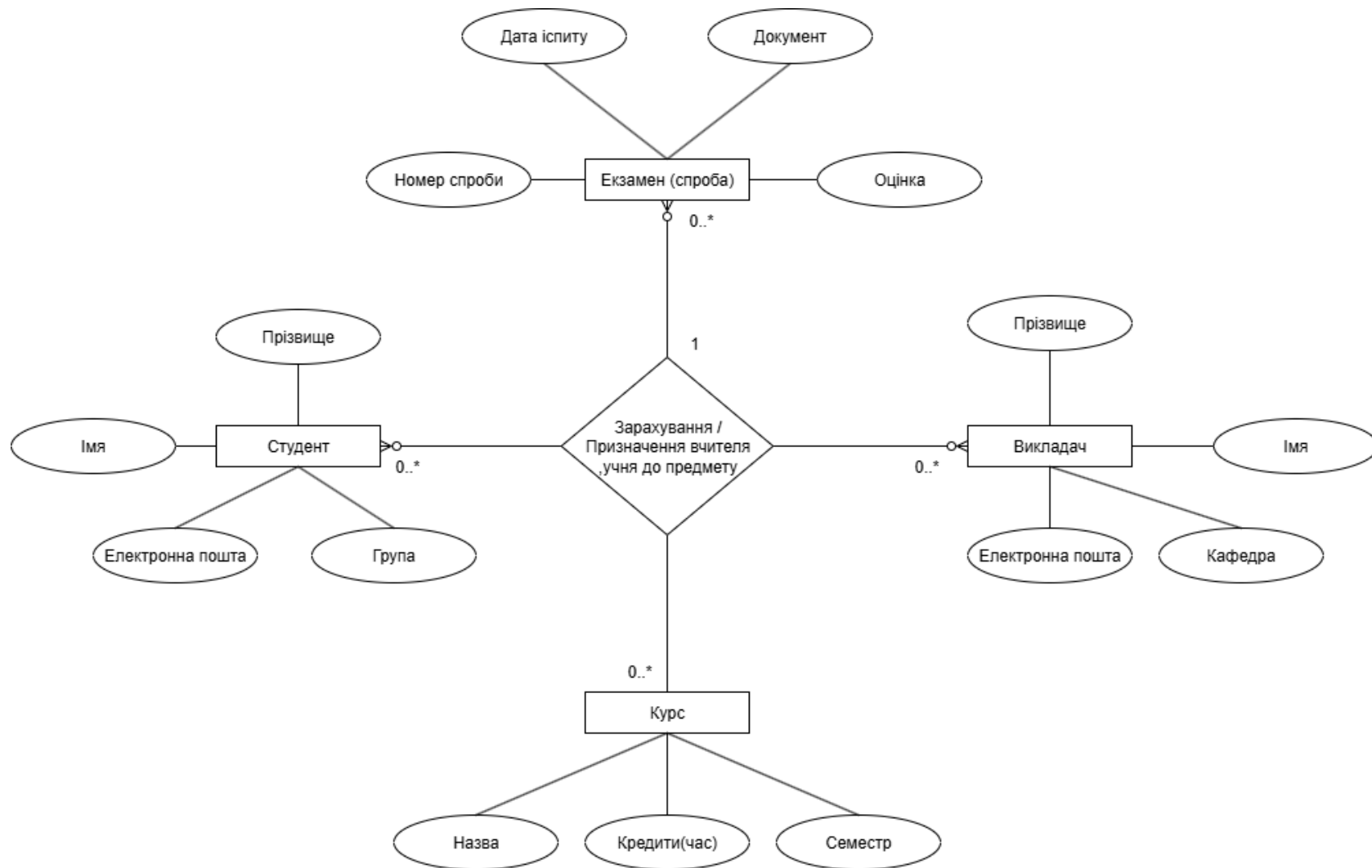


Рисунок 1 – ER-діаграма

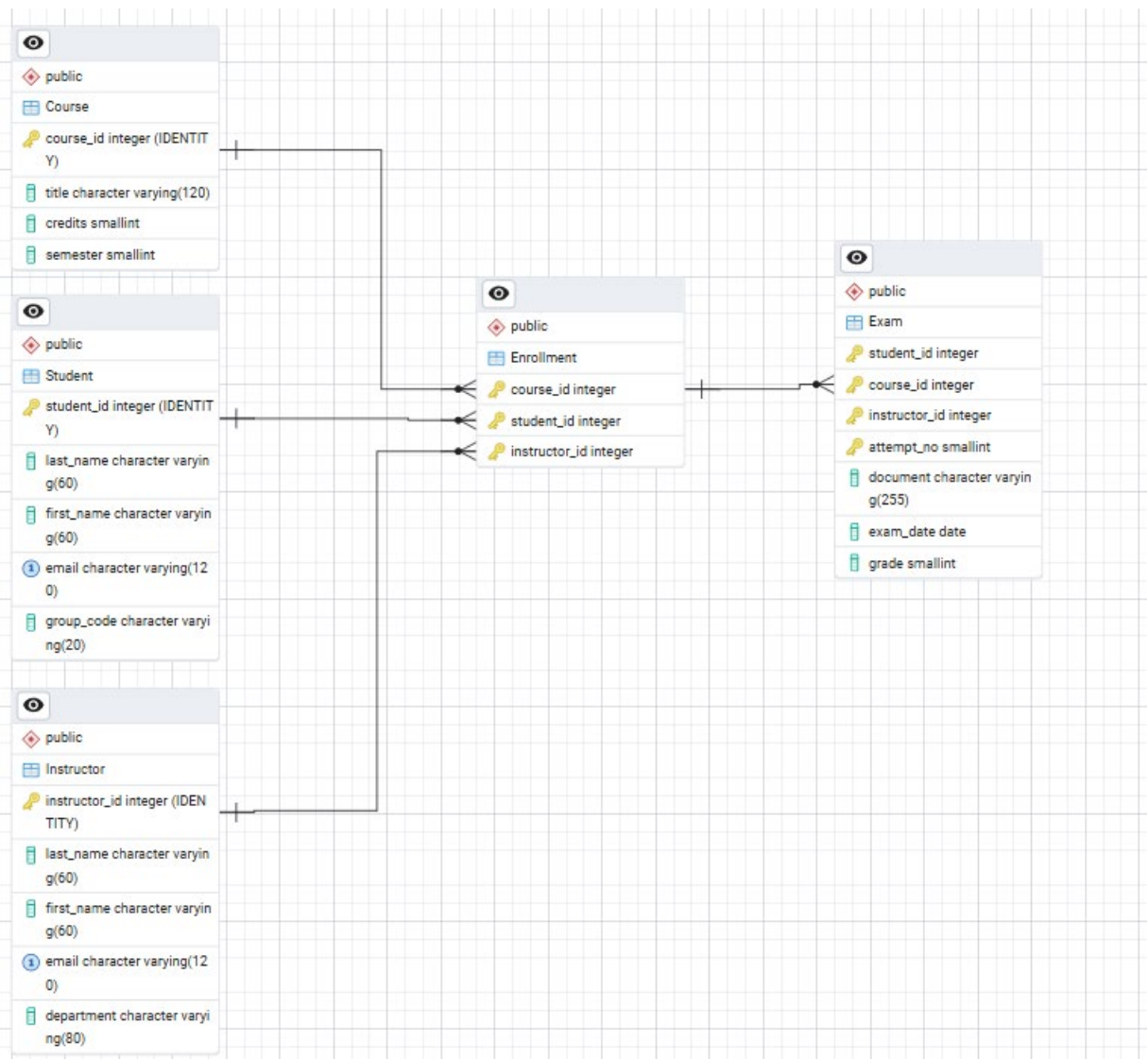


Рисунок 2– Схема бази даних у PostgreSQL

2.2. Схема меню користувача та використані технології

2.2.1. Схема меню користувача

ГОЛОВНЕ МЕНЮ

- 1) CRUD: Student
- 2) CRUD: Instructor
- 3) CRUD: Course
- 4) CRUD: Enrollment (трійки)
- 5) CRUD: Exam (спроби)
- 6) Генерація даних
- 7) Пошуки (3 запити + час)
- 0) Вихід

Опис пунктів головного меню

1) CRUD: Student

Підменю: Перегляд | Додати | Оновити | Видалити | Назад

- Перегляд: вивід перших 50 записів (ID, ПІБ, email, group_code).
- Додати/Оновити: введення та валідація типів (рядки, email як рядок), унікальності email контролюється БД.
- Видалити: контроль 1:N — якщо існують залежні записи в Enrollment/Exam, видалення блокується повідомленням (без виводу системної помилки).

2) CRUD: Instructor

Підменю: Перегляд | Додати | Оновити | Видалити | Назад

- Аналогічно до Student.
- Видалення: заборонене, якщо є залежні Enrollment/Exam (контроль 1:N до виконання SQL).

3) CRUD: Course

Підменю: Перегляд | Додати | Оновити | Видалити | Назад

- Валідація credits і semester як smallint.
- Видалення: заборонене за наявності залежних рядків у Enrollment/Exam.

4) CRUD: Enrollment (трійки)

Підменю: Перегляд | Додати | Оновити | Видалити | Назад

- Перегляд: список “трійок” із назвами курсу та ПІБ студента/викладача.
- Додати: перевірка існування усіх трьох батьківських ключів (Student, Course, Instructor) до вставки; FK-помилки перехоплюються.
- Оновити: не застосовується (PK — комбінація course_id, student_id, instructor_id), зміна “трійки” виконується як видалення та повторна вставка.
- Видалити: заборонено, якщо є залежні Exam (контроль 1:N).

5) CRUD: Exam (спроби)

Підменю: Перегляд | Додати | Оновити | Видалити | Назад

- Перегляд: список спроб із ПІБ, курсом, датою та оцінкою.
- Додати: перед вставкою перевіряється наявність батьківської трійки в Enrollment; доступна автонумерація attempt_no (наступний номер).
- Оновити: редагуються неключові поля document, exam_date, grade (РК лишається незмінним).
- Видалити: видалення однієї спроби за РК (student_id, course_id, instructor_id, attempt_no).

6) Генерація даних

Підменю: Student (N)|Instructor(~N/100)|Course (пул) | Enrollment | Exam | Масова генерація | Назад

- Уся генерація виконується виключно SQL (без циклів Python) з використанням generate_series() та random().
- Користувач вводить N; рекомендований порядок масової генерації: Student → Instructor → Course → Enrollment → Exam.
- Вимоги FK: спочатку батьківські таблиці, потім — підлеглі; дублікати унікальних значень і конфлікти РК/FK безпечно перехоплюються та не призводять до системних помилок у консолі.

7) Пошуки (3 запити + час виконання)

Підменю: Пошук 1 | Пошук 2 | Пошук 3 | Назад

- Пошук 1. Student–Exam–Course: фільтри — group_code, діапазон оцінок (grade BETWEEN), шаблон назви курсу (ILIKE), діапазон дат іспиту (exam_date BETWEEN).
- Пошук 2. Агрегація Exam за Course і Instructor: параметри — діапазон дат, мінімальна кількість спроб у групі (HAVING COUNT(*)). Повертаються attempts і avg_grade.
- Пошук 3. Хто (за останньою спробою) нижче порогу: параметри — semester, мінімальні credits, поріг; визначення береться за останньою спробою (останній exam_date, при рівності — найбільший attempt_no).
- Після кожного запиту виводиться час виконання (мс).

0) Вихід

Завершення програми

2.2.2. Мова програмування та використані бібліотеки

- Мова програмування: Python 3.11.9
- Бібліотеки:
- psycopg 3 — драйвер доступу до PostgreSQL; робота суто SQL, без ORM.
- python-dotenv — завантаження параметрів підключення з файлу .env (DATABASE_URL).
- Стандартна бібліотека: os (змінні середовища), time (вимір часу запитів у мс), datetime (валідація дат), typing (підказки типів).
- СУБД: PostgreSQL (робота з узгодженою схемою: "Student", "Instructor", "Course", "Enrollment", "Exam").
- Відлагодження SQL: PgAdmin 4 (за методичкою).

Перше деталізоване завдання

2.3. Вилучення запису батьківської таблиці та поведінка дочірньої (контроль 1:N)

Мета: Показати, що застосунок:

- дозволяє безпечно видаляти батьківський запис лише якщо немає залежних даних у підлеглих таблицях;
- забороняє видалення за наявності залежних рядків та не виводить системних помилок, а показує коректне повідомлення користувачу;
- пояснює причини неможливості видалення (обмеження цілісності, зв'язок 1:N).

2.3.1. Випадок А — видалення можливе (немає залежних даних)

1. Створити викладача без призначень у Enrollment.

```

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
Прізвище: Шевченко
Ім'я: Богдан
Email: b.shevchenko@uni.edu
Кафедра/департамент: Бази даних
[ІНФО] Додано instructor_id=4

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'instructor_id': 1, 'last_name': 'Коваленко', 'first_name': 'Олена', 'email': 'o.kovalenko@uni.edu', 'department': 'Інформаційні технології'}
{'instructor_id': 2, 'last_name': 'Іванов', 'first_name': 'Петро', 'email': 'p.ivanov@uni.edu', 'department': 'Математика'}
{'instructor_id': 3, 'last_name': 'Савчук', 'first_name': 'Дарія', 'email': 'd.savchuk@uni.edu', 'department': 'Фізика'}
{'instructor_id': 4, 'last_name': 'Шевченко', 'first_name': 'Богдан', 'email': 'b.shevchenko@uni.edu', 'department': 'Бази даних'}

```

2. Перевірити, що для цього instructor id немає рядків у Enrollment.

```

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко', 'i_first': 'Олена'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар', 's_first': 'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last': 'Ченко', 's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук', 'i_first': 'Дарія'}

```

3. Видалити викладача 2 → “Видалити”

```

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 4
ID викладача для видалення: 4
[ІНФО] Видалено рядків: 1

```

4. Повторно відкрити Enrollment

```

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко', 'i_first': 'Олена'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар', 's_first': 'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last': 'Ченко', 's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук', 'i_first': 'Дарія'}

```

Пояснення: Видалення дозволене, оскільки для батьківської сутності Instructor не існує жодного рядка в дочірній Enrollment (а відтак і в Exam). На рівні застосунку виконується попередня перевірка залежностей; системні помилки СУБД не виникають.

Лістинг 2.3.А (консоль):

```

=== Головне меню ===
1) CRUD: Student
2) CRUD: Instructor
3) CRUD: Course
4) CRUD: Enrollment (трійки)
5) CRUD: Exam (спроби)
6) Генерація даних
7) Пошуки (3 запити + час виконання)
0) Вихід
> 2

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
Прізвище: Шевченко
Ім'я: Богдан
Email: b.shevchenko@uni.edu
Кафедра/департамент: Бази даних
[ІНФО] Додано instructor_id=4

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'instructor_id': 1, 'last_name': 'Коваленко', 'first_name': 'Олена', 'email':
'o.kovalenko@uni.edu', 'department': 'Інформаційні технології'}
{'instructor_id': 2, 'last_name': 'Іванов', 'first_name': 'Петро', 'email':
'p.ivanov@uni.edu', 'department': 'Математика'}
{'instructor_id': 3, 'last_name': 'Савчук', 'first_name': 'Дарія', 'email':
'd.savchuk@uni.edu', 'department': 'Фізика'}
{'instructor_id': 4, 'last_name': 'Шевченко', 'first_name': 'Богдан', 'email':
'b.shevchenko@uni.edu', 'department': 'Бази даних'}

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 0

=== Головне меню ===
1) CRUD: Student
2) CRUD: Instructor
3) CRUD: Course
4) CRUD: Enrollment (трійки)
5) CRUD: Exam (спроби)
6) Генерація даних
7) Пошуки (3 запити + час виконання)
0) Вихід
> 4

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last': 'Сидоренко',
's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко', 'i_first': 'Олена'}

```

```

{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last': 'Сидоренко',
's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар', 's_first':
'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last': 'Ченко',
's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук', 'i_first': 'Дарія'}

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 0

=== Головне меню ===
1) CRUD: Student
2) CRUD: Instructor
3) CRUD: Course
4) CRUD: Enrollment (трійки)
5) CRUD: Ехам (спроби)
6) Генерація даних
7) Пошуки (3 запити + час виконання)
0) Вихід
> 2

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 4
ID викладача для видалення: 4
[ІНФО] Видалено рядків: 1

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 0

=== Головне меню ===
1) CRUD: Student
2) CRUD: Instructor
3) CRUD: Course
4) CRUD: Enrollment (трійки)
5) CRUD: Ехам (спроби)
6) Генерація даних
7) Пошуки (3 запити + час виконання)
0) Вихід
> 4

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last': 'Сидоренко',
's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко', 'i_first': 'Олена'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last': 'Сидоренко',
's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар', 's_first':
'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last': 'Ченко',
's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук', 'i_first': 'Дарія'}

```


2.3.2. Випадок Б — видалення неможливе (є залежні дані)

1. Обрати студента/викладача/курс, які вже фігурують у Enrollment (тобто є підлеглі рядки).

```
--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко', 'i_first': 'Олена'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last': 'Сидоренко', 's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар', 's_first': 'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first': 'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last': 'Ченко', 's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук', 'i_first': 'Дарія'}
```

2. Спроба видалити батьківський запис через відповідне меню CRUD.

```
--- Student ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 4
ID студента для видалення: 1
[ПОМИЛКА] Видалення заборонено: існують залежні рядки в Enrollment/Exam.
```

Пояснення: у моделі БД таблиця Enrollment має зовнішні ключі на Student/Course/Instructor (зв'язок 1:N). Наявність підлеглих рядків означає, що видалення батьківського об'єкта призведе до порушення цілісності (FK) або небажаного каскадного видалення даних.

Лістинг 2.3.Б (фрагмент контролера, що реалізує контроль 1:N перед видаленням):

```
# controller.py (Student → Видалити)
sid = self.v.ask_int("ID студента для видалення: ", 1)
dep = self.m.count_enrollments_by_student(sid)
if dep > 0:
    self.v.err("Видалення заборонено: існують залежні рядки в Enrollment/Exam.")
else:
    cnt = self.m.student_delete(sid)
    self.v.info(f"Видалено рядків: {cnt}")
```

Лістинг 2.3.Б (консоль):

[ІНФО] Підключення до БД виконано успішно.

=== Головне меню ===

- 1) **CRUD**: Student
 - 2) **CRUD**: Instructor
 - 3) **CRUD**: Course
 - 4) **CRUD**: Enrollment (трійки)
 - 5) **CRUD**: Exam (спроби)
 - 6) Генерація даних
 - 7) Пошуки (3 запити + час виконання)
 - 0) Вихід
- > 4

--- Enrollment (course_id, student_id, instructor_id) ---

- 1) Перегляд (перші 50)
 - 2) Додати
 - 3) Оновити
 - 4) Видалити
 - 0) Назад
- > 1
- ```
{'course_id': 1, 'title': 'Бази даних', 'student_id': 1, 's_last':
'Sидоренко', 's_first': 'Марія', 'instructor_id': 1, 'i_last': 'Коваленко',
'i_first': 'Олена'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 1, 's_last':
'Sидоренко', 's_first': 'Марія', 'instructor_id': 2, 'i_last': 'Іванов',
'i_first': 'Петро'}
{'course_id': 2, 'title': 'Алгоритми', 'student_id': 2, 's_last': 'Бондар',
's_first': 'Андрій', 'instructor_id': 2, 'i_last': 'Іванов', 'i_first':
'Петро'}
{'course_id': 3, 'title': 'Вища математика', 'student_id': 3, 's_last':
'Ченко', 's_first': 'Ілля', 'instructor_id': 3, 'i_last': 'Савчук',
'i_first': 'Дарія'}
```

--- Enrollment (course\_id, student\_id, instructor\_id) ---

- 1) Перегляд (перші 50)
  - 2) Додати
  - 3) Оновити
  - 4) Видалити
  - 0) Назад
- > 0

=== Головне меню ===

- 1) **CRUD**: Student
  - 2) **CRUD**: Instructor
  - 3) **CRUD**: Course
  - 4) **CRUD**: Enrollment (трійки)
  - 5) **CRUD**: Exam (спроби)
  - 6) Генерація даних
  - 7) Пошуки (3 запити + час виконання)
  - 0) Вихід
- > 1

--- Student ---

- 1) Перегляд (перші 50)
  - 2) Додати
  - 3) Оновити
  - 4) Видалити
  - 0) Назад
- > 4

**ID студента для видалення: 1**

**[ПОМИЛКА] Видалення заборонено:** існують залежні рядки в Enrollment/Exam.

## **2.4. Вставка в дочірню таблицю та поведінка за відсутнього батьківського запису**

**Мета пункту:** Показати обидва підходи:

- **Контроль на вході:** перевірка існування батьківського рядка перед вставкою;
- **Перехоплення помилки СУБД:** у випадку прямої спроби вставити невалідний FK — повідомлення.

### 2.4.1. Дочірня таблиця Enrollment — перевірка існування батьків (Student/Course/Instructor)

1. Обрати пункт меню 4) CRUD: Enrollment → “Додати”

```

=== Головне меню ===
1) CRUD: Student
2) CRUD: Instructor
3) CRUD: Course
4) CRUD: Enrollment (трійки)
5) CRUD: Exam (спроби)
6) Генерація даних
7) Пошуки (3 запити + час виконання)
0) Вихід
> 4

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
course_id:

```

2. Ввести, наприклад, course\_id=999999 (якого не існує), валідні student id і instructor id

```

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
course_id: 999999
student_id: 1
instructor_id: 1

```

3. Застосунок перевірить існування всіх трьох батьківських записів і не відправить INSERT, якщо чогось не вистачає або якщо данні фальшиві.

```

--- Enrollment (course_id, student_id, instructor_id) ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
course_id: 999999
student_id: 1
instructor_id: 1
[ПОМИЛКА] Немає такого course_id.

```

**Пояснення.** Застосунок реалізує перший із рекомендованих підходів — контроль при введенні. Це виключає помилки FK ще до звернення до СУБД і запобігає “системним” повідомленням.

### Лістинг 2.4.1 (фрагмент контролера):

```
controller.py (Enrollment → Додати)
cid = self.v.ask_int("course_id: ", 1)
sid = self.v.ask_int("student_id: ", 1)
iid = self.v.ask_int("instructor_id: ", 1)

if not self.m.course_get(cid):
 self.v.err("Немає такого course_id."); return
if not self.m.student_get(sid):
 self.v.err("Немає такого student_id."); return
if not self.m.instructor_get(iid):
 self.v.err("Немає такого instructor_id."); return

added = self.m.enrollment_create(cid, sid, iid)
self.v.info("Enrollment додано." if added == 1 else "Така трійка вже існує.")
```

### Лістинг 2.4.1 (консоль):

[ІНФО] Підключення до БД виконано успішно.

=== Головне меню ===

- 1) CRUD: Student
  - 2) CRUD: Instructor
  - 3) CRUD: Course
  - 4) CRUD: Enrollment (трійки)
  - 5) CRUD: Exam (спроби)
  - 6) Генерація даних
  - 7) Пошуки (3 запити + час виконання)
  - 0) Вихід
- > 4

--- Enrollment (course\_id, student\_id, instructor\_id) ---

- 1) Перегляд (перші 50)
  - 2) Додати
  - 3) Оновити
  - 4) Видалити
  - 0) Назад
- > 2

course\_id: 999999

student\_id: 1

instructor\_id: 1

[ПОМИЛКА] Немає такого course\_id.

### 2.4.2. Дочірня таблиця Exam — спроба вставки без існуючої трійки в Enrollment (демонстрація перехоплення помилки СУБД)

Тут навмисно демонструємо другий підхід— перехоплення серверної помилки.

У застосунку перед вставкою Exam ми теж перевіряємо наявність трійки (контроль на вході). Але нам потрібно перехопити перехоплення помилки. Це можна відтворити, **тимчасово** вимкнувши перевірку в контролері

```

Exam

def menu_exam(self):
 while True:
 ch = self.v.submenu_crud("Exam")
 if ch == "1":
 rows = self.m.exam_list()
 self.v.show_rows(rows)
 elif ch == "2":
 sid = self.v.ask_int("student_id: ", 1)
 cid = self.v.ask_int("course_id: ", 1)
 iid = self.v.ask_int("instructor_id: ", 1)
 # Перевірка наявності батьківської трійки
 if not self.m.enrollment_exists(cid, sid, iid):
 # self.v.err("Немає відповідної трійки в Enrollment. Спершу створіть Enrollment.")
 # continue
 # Пропонуємо авто-нумерацію спроб
 auto = self.v.confirm("Автоматично обрати наступний attempt_no? ")
 if auto:
 attempt_no = self.m.exam_get_next_attempt(sid, cid, iid)
 self.v.info(f"Встановлено attempt_no={attempt_no}")
 else:
 attempt_no = self.v.ask_smallint("attempt_no (smallint): ", 1)
 doc = self.v.ask_str("document (рядок/посилання): ")
 dt = self.v.ask_date_iso("exam_date (YYYY-MM-DD): ")
 grade = self.v.ask_smallint("grade (0..100): ", 0, 100)
 cnt = self.m.exam_create(sid, cid, iid, attempt_no, doc, dt, grade)
 self.v.info(f"Додано рядків: {cnt}")
```

Очікуваний консольний результат (коли FK-помилка доходить із СУБД до застосунку):

[ПОМИЛКА] Операцію відхилено: порушення зовнішнього ключа (foreign key).

```
--- Exam ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 2
student_id: 999
course_id: 999
instructor_id: 999
Автоматично обрати наступний attempt_no? [y/N]: y
[ІНФО] Встановлено attempt_no=1
document (рядок/посилання): 1.doc
exam_date (YYYY-MM-DD): 2025-10-13
grade (0..100): 100
[ПОМИЛКА] Операцію відхилено: порушення зовнішнього ключа (foreign key).
```

**Лістинг 2.4.2 (фрагмент загального перехоплення в Controller.run()):**

```
except psycopg.errors.ForeignKeyViolation:
 self.v.err("Операцію відхилено: порушення зовнішнього ключа (foreign
key).")
except psycopg.errors.UniqueViolation:
 self.v.err("Операцію відхилено: порушення унікальності (наприклад, email
вже існує).")
except psycopg.Error as e:
 self.v.err(f"Помилка БД (SQLSTATE={e.sqlstate}). Операцію не виконано.")
```

## Друге деталізоване завдання

### 2.5. Копії SQL-запитів, що ілюструють генерацію при визначених вхідних параметрах

**Мета:** Показати що генерація виконується виключно SQL-запитами з меню програми б) Генерація даних. Параметр(и) N вводяться з клавіатури. Вставка робиться пакетно в СУБД за допомогою `generate_series()`, `random()`, `unnest()` тощо.

#### 2.5.1. Student — масова генерація 100 000 записів

**Вхідні параметри:** N\_students = 100000

**Еквівалентний SQL:**

```
INSERT INTO "Student"(last_name, first_name, email, group_code)
SELECT
 'Last_' || gs::text,
 'First_' || gs::text,
 'student_' || gs::text || '@example.edu',
 'KP-' || (10 + (random()*20)::int)::text
FROM generate_series(1, 100000) AS gs;
```

**Очікуваний консольний підсумок:**

[ІНФО] Додано Student: 100000

```
--- Генерація ---
1) Student (N)
2) Instructor (~N/100)
3) Course (пул курсів)
4) Enrollment (із наявних)
5) Exam (із наявних)
6) Масова генерація (рекомендований порядок 1→2→3→4→5)
0) Назад
> 1
К-сть Student (напр. 100000): 100000
[ІНФО] Додано Student: 100000
```



### 2.5.2. Instructor — приблизно $N/100$ унікальних викладачів

**Вхідні параметри:** базове  $N = 100000 \rightarrow$  програма обчислює  
 $M_{\text{instructors}} = \max(1, N/100) = 1000$

#### Еквівалентний SQL:

```
INSERT INTO "Instructor"(last_name, first_name, email, department)
SELECT
 'InstrL_' || gs::text,
 'InstrF_' || gs::text,
 'instructor_' || gs::text || '@univ.edu',
 'Dept_' || (1 + (random()*10)::int)::text
FROM generate_series(1, 1000) AS gs;
```

#### Очікуваний консольний підсумок:

[ІНФО] Додано Instructor: 1000

```
--- Генерація ---
1) Student (N)
2) Instructor (~N/100)
3) Course (пул курсів)
4) Enrollment (із наявних)
5) Exam (із наявних)
6) Масова генерація (рекомендований порядок 1→2→3→4→5)
0) Назад
> 2
Базове N (для ~N/100 викладачів): 100000
[ІНФО] Додано Instructor: 1000
```

### 2.5.3. Course — фіксований пул курсів

**Вхідні параметри:** не потрібні

#### Еквівалентний SQL:

```
INSERT INTO "Course"(title, credits, semester)
SELECT t,
 (1 + (random()*5)::int)::smallint,
 (1 + (random()*8)::int)::smallint
FROM unnest(ARRAY[
 'Databases', 'Algorithms', 'Discrete Math', 'Networks',
 'OS', 'AI', 'ML', 'SE', 'Web'
]) AS t;
```

#### Очікуваний консольний підсумок:

[ІНФО] Додано Course: 9

```
--- Генерація ---
1) Student (N)
2) Instructor (~N/100)
3) Course (пул курсів)
4) Enrollment (із наявних)
5) Exam (із наявних)
6) Масова генерація (рекомендований порядок 1→2→3→4→5)
0) Назад
> 3
[ІНФО] Додано Course: 9
```

### 2.5.4. Enrollment — генерація трійок (Student–Course–Instructor)

Щоб отримати  $\approx 120\ 000$  призначень, обираємо базове  $N_{\text{enr}} = 1200$  у Формула, яку використовує програма, створює приблизно  $N_{\text{enr}} * 100$  комбінацій:

- вибирається  $N_{\text{enr}}/2$  студентів, 10 курсів, 20 викладачів;
- картезіанський добуток дає  $(N_{\text{enr}}/2)*10*20 = N_{\text{enr}}*100$ .

**Вхідні параметри:**  $N_{\text{enr}} = 1200 \Rightarrow$  очікувано  $\approx 120\ 000$  рядків у Enrollment.

#### Еквівалентний SQL:

```
WITH s AS (SELECT student_id FROM "Student" ORDER BY random() LIMIT
GREATEST(1, 1200/2)),
 c AS (SELECT course_id FROM "Course" ORDER BY random() LIMIT 10),
 i AS (SELECT instructor_id FROM "Instructor" ORDER BY random()
LIMIT 20)
INSERT INTO "Enrollment"(course_id, student_id, instructor_id)
SELECT c.course_id, s.student_id, i.instructor_id
FROM s CROSS JOIN c CROSS JOIN i
ON CONFLICT DO NOTHING;
```

**Примітка.** ON CONFLICT DO NOTHING гарантує відсутність дублювань РК (однакова трійка не вставляється двічі). Реальна кількість вставлених рядків може бути трохи меншою за 120 000 через конфлікти або нестачу унікальних комбінацій — це нормально.

#### Очікуваний консольний підсумок (приблизно):

[ІНФО] Додано Enrollment: 120000

```
--- Генерація ---
1) Student (N)
2) Instructor (~N/100)
3) Course (пул курсів)
4) Enrollment (із наявних)
5) Exam (із наявних)
6) Масова генерація (рекомендований порядок 1→2→3→4→5)
0) Назад
> 4
Базове N (для масштабу Enrollment): 1200
[ІНФО] Додано Enrollment: 108000
```

Як бачимо реальних значень трохи менше, про що згадувалося раніше, а саме 108000

### 2.5.5. Exam — генерація спроб іспитів із випадковими датами та оцінками

Кожній вибраній трійці робиться **1–3 спроби**. Щоб отримати **≈100–150 тис.** записів, беремо  $N_{\text{exam}} = 50000$  у  $6 \rightarrow 5$ : буде обрано 50 000 різних трійок, для кожної — 1..3 спроби (очікувано близько 100–150 тис. рядків у Exam)

**Вхідні параметри:**  $N_{\text{exam}} = 50000$

#### Еквівалентний SQL:

```
WITH e AS (
 SELECT course_id, student_id, instructor_id
 FROM "Enrollment"
 ORDER BY random()
 LIMIT 50000
),
attempts AS (
 SELECT e.*, gs AS attempt_no
 FROM e
 JOIN generate_series(1, (1 + (random()*3)::int)) AS gs ON TRUE
)
INSERT INTO "Exam"(student_id, course_id, instructor_id, attempt_no,
document, exam_date, grade)
SELECT
 student_id, course_id, instructor_id, attempt_no,
 'doc_' || md5(random()::text),
 (CURRENT_DATE - ((random()*1095)::int)::text::interval)::date, --
 випадково в межах ~3 років
 (random()*100)::int
FROM attempts
ON CONFLICT DO NOTHING;
```

#### Очікуваний консольний підсумок (приблизно):

[ІНФО] Додано Exam: 120000

```
--- Генерація ---
1) Student (N)
2) Instructor (~N/100)
3) Course (пул курсів)
4) Enrollment (із наявних)
5) Exam (із наявних)
6) Масова генерація (рекомендований порядок 1→2→3→4→5)
0) Назад
> 5
Базове N (скільки трійок взяти для Exam): 50000
[ІНФО] Додано Exam: 150000
```

Вийшло трохи більше очікуваного

## 2.6. Копії екрана (ілюстрації) з фрагментами згенерованих даних таблиць(а також фото з pgAdmin)

Після кожного кроку генерації даних у програмі відкриваємо відповідне CRUD-меню → **Перегляд (перші 50)**, щоб зафіксувати результат. Для підтвердження великого обсягу даних наведено також скріншоти з pgAdmin4 (вивід у консолі обмежено до перших 50 рядків з міркувань читабельності).

### 2.6.1. Student — вибірка перших записів

```

--- Student ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'student_id': 33, 'last_name': 'Last_1', 'first_name': 'First_1', 'email': 'student_1@example.edu', 'group_code': 'KP-29'}
{'student_id': 34, 'last_name': 'Last_2', 'first_name': 'First_2', 'email': 'student_2@example.edu', 'group_code': 'KP-25'}
{'student_id': 35, 'last_name': 'Last_3', 'first_name': 'First_3', 'email': 'student_3@example.edu', 'group_code': 'KP-11'}
{'student_id': 36, 'last_name': 'Last_4', 'first_name': 'First_4', 'email': 'student_4@example.edu', 'group_code': 'KP-14'}
{'student_id': 37, 'last_name': 'Last_5', 'first_name': 'First_5', 'email': 'student_5@example.edu', 'group_code': 'KP-17'}
{'student_id': 38, 'last_name': 'Last_6', 'first_name': 'First_6', 'email': 'student_6@example.edu', 'group_code': 'KP-22'}
{'student_id': 39, 'last_name': 'Last_7', 'first_name': 'First_7', 'email': 'student_7@example.edu', 'group_code': 'KP-21'}
{'student_id': 40, 'last_name': 'Last_8', 'first_name': 'First_8', 'email': 'student_8@example.edu', 'group_code': 'KP-26'}
{'student_id': 41, 'last_name': 'Last_9', 'first_name': 'First_9', 'email': 'student_9@example.edu', 'group_code': 'KP-29'}
{'student_id': 42, 'last_name': 'Last_10', 'first_name': 'First_10', 'email': 'student_10@example.edu', 'group_code': 'KP-23'}
{'student_id': 43, 'last_name': 'Last_11', 'first_name': 'First_11', 'email': 'student_11@example.edu', 'group_code': 'KP-25'}
{'student_id': 44, 'last_name': 'Last_12', 'first_name': 'First_12', 'email': 'student_12@example.edu', 'group_code': 'KP-25'}
{'student_id': 45, 'last_name': 'Last_13', 'first_name': 'First_13', 'email': 'student_13@example.edu', 'group_code': 'KP-29'}
{'student_id': 46, 'last_name': 'Last_14', 'first_name': 'First_14', 'email': 'student_14@example.edu', 'group_code': 'KP-15'}
{'student_id': 47, 'last_name': 'Last_15', 'first_name': 'First_15', 'email': 'student_15@example.edu', 'group_code': 'KP-17'}

```

|   | student_id<br>[PK] integer | last_name<br>character varying (60) | first_name<br>character varying (60) | email<br>character varying (120) | group_code<br>character varying (20) |
|---|----------------------------|-------------------------------------|--------------------------------------|----------------------------------|--------------------------------------|
| 1 | 33                         | Last_1                              | First_1                              | student_1@example.edu            | KP-29                                |
| 2 | 34                         | Last_2                              | First_2                              | student_2@example.edu            | KP-25                                |
| 3 | 35                         | Last_3                              | First_3                              | student_3@example.edu            | KP-11                                |
| 4 | 36                         | Last_4                              | First_4                              | student_4@example.edu            | KP-14                                |
| 5 | 37                         | Last_5                              | First_5                              | student_5@example.edu            | KP-17                                |
| 6 | 38                         | Last_6                              | First_6                              | student_6@example.edu            | KP-22                                |
| 7 | 39                         | Last_7                              | First_7                              | student_7@example.edu            | KP-21                                |
| 8 | 40                         | Last_8                              | First_8                              | student_8@example.edu            | KP-26                                |
| 9 | 41                         | Last_9                              | First_9                              | student_9@example.edu            | KP-29                                |

Total rows: 100000    Query complete 00:00:00.072

## 2.6.2. Instructor — вибірка перших записів

```

--- Instructor ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'instructor_id': 8, 'last_name': 'InstrL_1', 'first_name': 'InstrF_1', 'email': 'instructor_1@univ.edu', 'department': 'Dept_9'}
{'instructor_id': 9, 'last_name': 'InstrL_2', 'first_name': 'InstrF_2', 'email': 'instructor_2@univ.edu', 'department': 'Dept_3'}
{'instructor_id': 10, 'last_name': 'InstrL_3', 'first_name': 'InstrF_3', 'email': 'instructor_3@univ.edu', 'department': 'Dept_6'}
{'instructor_id': 11, 'last_name': 'InstrL_4', 'first_name': 'InstrF_4', 'email': 'instructor_4@univ.edu', 'department': 'Dept_9'}
{'instructor_id': 12, 'last_name': 'InstrL_5', 'first_name': 'InstrF_5', 'email': 'instructor_5@univ.edu', 'department': 'Dept_6'}
{'instructor_id': 13, 'last_name': 'InstrL_6', 'first_name': 'InstrF_6', 'email': 'instructor_6@univ.edu', 'department': 'Dept_11'}
{'instructor_id': 14, 'last_name': 'InstrL_7', 'first_name': 'InstrF_7', 'email': 'instructor_7@univ.edu', 'department': 'Dept_5'}
{'instructor_id': 15, 'last_name': 'InstrL_8', 'first_name': 'InstrF_8', 'email': 'instructor_8@univ.edu', 'department': 'Dept_5'}
{'instructor_id': 16, 'last_name': 'InstrL_9', 'first_name': 'InstrF_9', 'email': 'instructor_9@univ.edu', 'department': 'Dept_9'}
{'instructor_id': 17, 'last_name': 'InstrL_10', 'first_name': 'InstrF_10', 'email': 'instructor_10@univ.edu', 'department': 'Dept_9'}
{'instructor_id': 18, 'last_name': 'InstrL_11', 'first_name': 'InstrF_11', 'email': 'instructor_11@univ.edu', 'department': 'Dept_2'}
{'instructor_id': 19, 'last_name': 'InstrL_12', 'first_name': 'InstrF_12', 'email': 'instructor_12@univ.edu', 'department': 'Dept_5'}
{'instructor_id': 20, 'last_name': 'InstrL_13', 'first_name': 'InstrF_13', 'email': 'instructor_13@univ.edu', 'department': 'Dept_7'}
{'instructor_id': 21, 'last_name': 'InstrL_14', 'first_name': 'InstrF_14', 'email': 'instructor_14@univ.edu', 'department': 'Dept_2'}
{'instructor_id': 22, 'last_name': 'InstrL_15', 'first_name': 'InstrF_15', 'email': 'instructor_15@univ.edu', 'department': 'Dept_3'}

```

| Showing rows: 1 to 1000 Page No: 1           |                               |                                     |                                      |                                  |                                      |
|----------------------------------------------|-------------------------------|-------------------------------------|--------------------------------------|----------------------------------|--------------------------------------|
|                                              | instructor_id<br>[PK] integer | last_name<br>character varying (60) | first_name<br>character varying (60) | email<br>character varying (120) | department<br>character varying (80) |
| 1                                            | 8                             | InstrL_1                            | InstrF_1                             | instructor_1@univ.edu            | Dept_9                               |
| 2                                            | 9                             | InstrL_2                            | InstrF_2                             | instructor_2@univ.edu            | Dept_3                               |
| 3                                            | 10                            | InstrL_3                            | InstrF_3                             | instructor_3@univ.edu            | Dept_6                               |
| 4                                            | 11                            | InstrL_4                            | InstrF_4                             | instructor_4@univ.edu            | Dept_9                               |
| 5                                            | 12                            | InstrL_5                            | InstrF_5                             | instructor_5@univ.edu            | Dept_6                               |
| 6                                            | 13                            | InstrL_6                            | InstrF_6                             | instructor_6@univ.edu            | Dept_11                              |
| 7                                            | 14                            | InstrL_7                            | InstrF_7                             | instructor_7@univ.edu            | Dept_5                               |
| 8                                            | 15                            | InstrL_8                            | InstrF_8                             | instructor_8@univ.edu            | Dept_5                               |
| 9                                            | 16                            | InstrL_9                            | InstrF_9                             | instructor_9@univ.edu            | Dept_9                               |
| Total rows: 1000 Query complete 00:00:00.055 |                               |                                     |                                      |                                  |                                      |

## 2.6.3. Course — вибірка перших записів

```

--- Course ---
1) Перегляд (перші 50)
2) Додати
3) Оновити
4) Видалити
0) Назад
> 1
{'course_id': 31, 'title': 'Databases', 'credits': 3, 'semester': 8}
{'course_id': 32, 'title': 'Algorithms', 'credits': 3, 'semester': 7}
{'course_id': 33, 'title': 'Discrete Math', 'credits': 5, 'semester': 8}
{'course_id': 34, 'title': 'Networks', 'credits': 4, 'semester': 7}
{'course_id': 35, 'title': 'OS', 'credits': 4, 'semester': 6}
{'course_id': 36, 'title': 'AI', 'credits': 5, 'semester': 4}
{'course_id': 37, 'title': 'ML', 'credits': 2, 'semester': 6}
{'course_id': 38, 'title': 'SE', 'credits': 2, 'semester': 1}
{'course_id': 39, 'title': 'Web', 'credits': 4, 'semester': 4}

```

| Showing rows: 1 to 9 Page No: |                           |                                  |                     |                      |
|-------------------------------|---------------------------|----------------------------------|---------------------|----------------------|
|                               | course_id<br>[PK] integer | title<br>character varying (120) | credits<br>smallint | semester<br>smallint |
| 1                             | 31                        | Databases                        | 3                   | 8                    |
| 2                             | 32                        | Algorithms                       | 3                   | 7                    |
| 3                             | 33                        | Discrete Math                    | 5                   | 8                    |
| 4                             | 34                        | Networks                         | 4                   | 7                    |
| 5                             | 35                        | OS                               | 4                   | 6                    |
| 6                             | 36                        | AI                               | 5                   | 4                    |
| 7                             | 37                        | ML                               | 2                   | 6                    |
| 8                             | 38                        | SE                               | 2                   | 1                    |
| 9                             | 39                        | Web                              | 4                   | 4                    |

2.6.4. Enrollment — вибірка перших записів

--- Enrollment (course\_id, student\_id, instructor\_id) ---  
1) Перегляд (перші 50)  
2) Додати  
3) Оновити  
4) Видалити  
0) Назад  
> 1  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 19, 'i\_last': 'Instrl\_12', 'i\_first': 'InstrF\_12'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 30, 'i\_last': 'Instrl\_23', 'i\_first': 'InstrF\_23'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 42, 'i\_last': 'Instrl\_35', 'i\_first': 'InstrF\_35'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 56, 'i\_last': 'Instrl\_49', 'i\_first': 'InstrF\_49'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 88, 'i\_last': 'Instrl\_81', 'i\_first': 'InstrF\_81'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 325, 'i\_last': 'Instrl\_318', 'i\_first': 'InstrF\_318'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 354, 'i\_last': 'Instrl\_347', 'i\_first': 'InstrF\_347'}

Showing rows: 1 to 1000

Page No: 1

|   | course_id<br>[PK] integer | student_id<br>[PK] integer | instructor_id<br>[PK] integer |
|---|---------------------------|----------------------------|-------------------------------|
| 1 | 31                        | 106                        | 19                            |
| 2 | 31                        | 106                        | 30                            |
| 3 | 31                        | 106                        | 42                            |
| 4 | 31                        | 106                        | 56                            |
| 5 | 31                        | 106                        | 88                            |
| 6 | 31                        | 106                        | 325                           |
| 7 | 31                        | 106                        | 354                           |
| 8 | 31                        | 106                        | 553                           |
| 9 | 31                        | 106                        | 555                           |

Total rows: 108000    Query complete 00:00:00.050

2.6.5. Exam — вибірка перших записів

--- Enrollment (course\_id, student\_id, instructor\_id) ---  
1) Перегляд (перші 50)  
2) Додати  
3) Оновити  
4) Видалити  
0) Назад  
> 1  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 19, 'i\_last': 'Instrl\_12', 'i\_first': 'InstrF\_12'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 30, 'i\_last': 'Instrl\_23', 'i\_first': 'InstrF\_23'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 42, 'i\_last': 'Instrl\_35', 'i\_first': 'InstrF\_35'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 56, 'i\_last': 'Instrl\_49', 'i\_first': 'InstrF\_49'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 88, 'i\_last': 'Instrl\_81', 'i\_first': 'InstrF\_81'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 325, 'i\_last': 'Instrl\_318', 'i\_first': 'InstrF\_318'}  
{'course\_id': 31, 'title': 'Databases', 'student\_id': 106, 's\_last': 'Last\_74', 's\_first': 'First\_74', 'instructor\_id': 354, 'i\_last': 'Instrl\_347', 'i\_first': 'InstrF\_347'}

Showing rows: 1 to 1000

Page No: 1

|   | student_id<br>[PK] integer | course_id<br>[PK] integer | instructor_id<br>[PK] integer | attempt_no<br>[PK] smallint | document<br>character varying (255)   | exam_date<br>date | grade<br>smallint |
|---|----------------------------|---------------------------|-------------------------------|-----------------------------|---------------------------------------|-------------------|-------------------|
| 1 | 106                        | 31                        | 30                            | 1                           | doc_40d078f3b5fbde3fe3c3a95a071656... | 2025-10-12        | 93                |
| 2 | 106                        | 31                        | 30                            | 2                           | doc_8c853b6fb48800de1cd5efbc5e334f... | 2025-10-12        | 80                |
| 3 | 106                        | 31                        | 30                            | 3                           | doc_aa39a810d71a5b4043287155b1c59...  | 2025-10-12        | 30                |
| 4 | 106                        | 31                        | 42                            | 1                           | doc_4080e5c6c30533da5e7a9a82139d8...  | 2025-10-12        | 17                |
| 5 | 106                        | 31                        | 42                            | 2                           | doc_d9126baed6f22c701df3364b7db8c...  | 2025-10-12        | 82                |
| 6 | 106                        | 31                        | 42                            | 3                           | doc_cb6a31a2fc661d45c31cfaeeceb623... | 2025-10-12        | 67                |
| 7 | 106                        | 31                        | 56                            | 1                           | doc_f760d4a3f3654726b8343e54ce513...  | 2025-10-12        | 39                |
| 8 | 106                        | 31                        | 56                            | 2                           | doc_16b2101f5fa215d07e44c6bb0f64f7... | 2025-10-12        | 66                |
| 9 | 106                        | 31                        | 56                            | 3                           | doc_68532ca70a35b4ee6eda12d9ca79e...  | 2025-10-12        | 57                |

Total rows: 150000    Query complete 00:00:00.127

## Третє деталізоване завдання

### 2.7. Ілюстрації введення пошукового запиту та результатів виконання

Всі пошуки запускаються з меню: **Головне меню → 7) Пошуки (3 запити + час)**

#### 2.7.1. Пошук 1 — фільтр за групою, діапазоном оцінок, шаблоном назви курсу та діапазоном дат

Послідовність у програмі:

1. 7 → 1
2. Ввести параметри :
  - Група: КР-17
  - Мін. оцінка: 60
  - Макс. оцінка: 95
  - Шаблон назви курсу: Data (програма перетворить на '%Data%')
  - Дата від: 2025-10-10
  - Дата до: 2025-10-15
3. Програма виведе результати у вигляді списку рядків (стовпці: student\_id, last\_name, first\_name, title, attempt\_no, exam\_date, grade) і час виконання в мілісекундах.

```

--- Пошуки ---
1) За групою + діапазон оцінок + шаблон курсу + діапазон дат
2) Середній бал/кількість спроб по курсах і викладачах за датами
3) Хто провалив при заданому семестрі/кредитах (мін. бал нижче порогу)
0) Назад
> 1
Група (напр. КР-21): КР-17
Мін. оцінка: 60
Макс. оцінка: 95
Шаблон назви курсу (напр. DB або %Data%): Data
Дата від (YYYY-MM-DD): 2025-10-10
Дата до (YYYY-MM-DD): 2025-10-15
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 3, "exam_date": datetime.date(2025, 10, 12), "grade": 81}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 2, "exam_date": datetime.date(2025, 10, 12), "grade": 74}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 1, "exam_date": datetime.date(2025, 10, 12), "grade": 83}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 3, "exam_date": datetime.date(2025, 10, 12), "grade": 87}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 2, "exam_date": datetime.date(2025, 10, 12), "grade": 80}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 1, "exam_date": datetime.date(2025, 10, 12), "grade": 81}
{"student_id": 17625, "last_name": "Last_17625", "first_name": "First_17625", "title": "Databases", "attempt_no": 1, "exam_date": datetime.date(2025, 10, 12), "grade": 86}

```

### 2.7.2. Пошук 2 — середній бал і кількість спроб по курсах та викладачах за діапазоном дат

#### Послідовність у програмі:

1. 7 → 2
2. Ввести параметри :
  - Дата від: 2025-10-10
  - Дата до: 2025-10-15
  - Мін. кількість спроб у групі: 890
3. Програма виведе агреговану таблицю (стовпці: title, instructor, attempts, avg\_grade) та час виконання.

```

--- Пошуки ---
1) За групою + діапазон оцінок + шаблон курсу + діапазон дат
2) Середній бал/кількість спроб по курсах і викладачах за датами
3) Хто провалив при заданому семестрі/кредитах (мін. бал нижче порогу)
0) Назад
> 2
Дата від (YYYY-MM-DD): 2025-10-10
Дата до (YYYY-MM-DD): 2025-10-15
Мін. кількість спроб у групі: 890
{'title': 'Web', 'instructor': 'InstrL_673 InstrF_673', 'attempts': 894, 'avg_grade': Decimal('51.8')}
{'title': 'SE', 'instructor': 'InstrL_343 InstrF_343', 'attempts': 900, 'avg_grade': Decimal('51.0')}
{'title': 'Web', 'instructor': 'InstrL_300 InstrF_300', 'attempts': 909, 'avg_grade': Decimal('50.2')}
{'title': 'ML', 'instructor': 'InstrL_415 InstrF_415', 'attempts': 912, 'avg_grade': Decimal('50.1')}
{'title': 'AI', 'instructor': 'InstrL_844 InstrF_844', 'attempts': 891, 'avg_grade': Decimal('49.9')}
{'title': 'AI', 'instructor': 'InstrL_415 InstrF_415', 'attempts': 900, 'avg_grade': Decimal('49.4')}
[INFO] Час виконання: 293.9 мс

```

### 2.7.3. Пошук 3 — студенти, які за останньою спробою нижче порогу при заданому семестрі/кредитах

#### Послідовність у програмі:

1. 7 → 3
2. Ввести параметри :
  - Семестр: 3
  - Мін. кредити: 3
  - Поріг "провалу": 60
3. Програма виведе таблицю (стовпці: student\_id, last\_name, first\_name, title, semester, credits, min\_grade) і час виконання.

```

datetime.date(2025, 10, 12))
{'student_id': 30215, 'last_name': 'Last_30215', 'first_name': 'First_30215', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
{'student_id': 43393, 'last_name': 'Last_43393', 'first_name': 'First_43393', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
{'student_id': 84352, 'last_name': 'Last_84352', 'first_name': 'First_84352', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
{'student_id': 44593, 'last_name': 'Last_44593', 'first_name': 'First_44593', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
{'student_id': 12951, 'last_name': 'Last_12951', 'first_name': 'First_12951', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
{'student_id': 23070, 'last_name': 'Last_23070', 'first_name': 'First_23070', 'title': 'Networks', 'semester': 3, 'credits': 5, 'last_grade': 94, 'last_exam_date': datetime.date(2025, 10, 12)}
[INFO] Час виконання: 141.5 мс

```



## 2.8. Копії SQL-запитів із зазначеними початковими параметрами

Нижче — ідентичні SQL, які виконує застосунок (у коді вони параметризовані), та еквіваленти з підставленими значеннями з прикладів п.2.7.

### 2.8.1. Пошук 1 — параметризований SQL

```
SELECT s.student_id, s.last_name, s.first_name, c.title,
 ex.attempt_no, ex.exam_date, ex.grade
FROM "Exam" ex
JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
JOIN "Student" s ON s.student_id = ex.student_id
JOIN "Course" c ON c.course_id = ex.course_id
WHERE s.group_code = %s
 AND ex.grade BETWEEN %s AND %s
 AND c.title ILIKE %s
 AND ex.exam_date BETWEEN %s AND %s
ORDER BY s.last_name, s.first_name, ex.exam_date DESC;
```

#### З підставленими параметрами (приклад з п.2.7.1):

```
SELECT s.student_id, s.last_name, s.first_name, c.title,
 ex.attempt_no, ex.exam_date, ex.grade
FROM "Exam" ex
JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
JOIN "Student" s ON s.student_id = ex.student_id
JOIN "Course" c ON c.course_id = ex.course_id
WHERE s.group_code = 'KP-17'
 AND ex.grade BETWEEN 60 AND 95
 AND c.title ILIKE '%Data%'
 AND ex.exam_date BETWEEN DATE '2025-10-10' AND DATE '2025-10-15'
ORDER BY s.last_name, s.first_name, ex.exam_date DESC;
```

## 2.8.2. Пошук 2 — параметризований SQL

```
SELECT c.title,
 i.last_name || ' ' || i.first_name AS instructor,
 COUNT(*) AS attempts,
 ROUND(AVG(ex.grade),1) AS avg_grade
FROM "Exam" ex
JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
JOIN "Course" c ON c.course_id = ex.course_id
JOIN "Instructor" i ON i.instructor_id = ex.instructor_id
WHERE ex.exam_date BETWEEN %s AND %s
GROUP BY c.title, i.last_name, i.first_name
HAVING COUNT(*) >= %s
ORDER BY avg_grade DESC;
```

### З підставленими параметрами (приклад з п.2.7.2):

```
SELECT c.title,
 i.last_name || ' ' || i.first_name AS instructor,
 COUNT(*) AS attempts,
 ROUND(AVG(ex.grade),1) AS avg_grade
FROM "Exam" ex
JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
JOIN "Course" c ON c.course_id = ex.course_id
JOIN "Instructor" i ON i.instructor_id = ex.instructor_id
WHERE ex.exam_date BETWEEN DATE '2025-10-10' AND DATE '2025-10-15'
GROUP BY c.title, i.last_name, i.first_name
HAVING COUNT(*) >= 890
ORDER BY avg_grade DESC;
```

### 2.8.3. Пошук 3 — параметризований SQL (як у коді)

```

WITH last_attempt AS (
 SELECT
 ex.student_id, ex.course_id, ex.instructor_id,
 ex.attempt_no, ex.document, ex.exam_date, ex.grade,
 ROW_NUMBER() OVER (
 PARTITION BY ex.student_id, ex.course_id, ex.instructor_id
 ORDER BY ex.exam_date DESC, ex.attempt_no DESC
) AS rn
 FROM "Exam" ex
)
SELECT
 s.student_id, s.last_name, s.first_name,
 c.title, c.semester, c.credits,
 la.grade AS last_grade,
 la.exam_date AS last_exam_date
FROM last_attempt la
JOIN "Enrollment" en
 ON en.student_id = la.student_id
 AND en.course_id = la.course_id
 AND en.instructor_id = la.instructor_id
JOIN "Student" s ON s.student_id = la.student_id
JOIN "Course" c ON c.course_id = la.course_id
WHERE la.rn = 1
 AND c.semester = %s
 AND c.credits >= %s
 AND la.grade < %s
ORDER BY last_grade ASC, last_exam_date DESC;

```

### 3 підставленими параметрами (приклад з п.2.7.3):

```

WITH last_attempt AS (
 SELECT
 ex.student_id, ex.course_id, ex.instructor_id,
 ex.attempt_no, ex.document, ex.exam_date, ex.grade,
 ROW_NUMBER() OVER (
 PARTITION BY ex.student_id, ex.course_id, ex.instructor_id
 ORDER BY ex.exam_date DESC, ex.attempt_no DESC
) AS rn
 FROM "Exam" ex
)
SELECT
 s.student_id, s.last_name, s.first_name,
 c.title, c.semester, c.credits,
 la.grade AS last_grade,
 la.exam_date AS last_exam_date
FROM last_attempt la
JOIN "Enrollment" en
 ON en.student_id = la.student_id
 AND en.course_id = la.course_id
 AND en.instructor_id = la.instructor_id
JOIN "Student" s ON s.student_id = la.student_id
JOIN "Course" c ON c.course_id = la.course_id
WHERE la.rn = 1
 AND c.semester = 3
 AND c.credits >= 3
 AND la.grade < 60
ORDER BY last_grade ASC, last_exam_date DESC;

```

## Четверте деталізоване завдання

### 2.9. Ілюстрації коду модуля Model (MVC) та короткий опис функцій

#### 2.9.1. Призначення модуля Model у шаблоні MVC

Модуль Model інкапсулює доступ до даних: підключення до PostgreSQL, виконання тільки SQL-запитів (без ORM), транзакції та повернення результатів у зручному вигляді для рівнів Controller/View. Уся логіка взаємодії з користувачем, валідації вводу та повідомлення про помилки — поза Model (у Controller/View).

```
model.py
Увесь доступ до БД

import psycopg
from psycopg.rows import dict_row
```

#### 2.9.2. Підключення, транзакції, формат рядків

- Підключення створюється через приватний метод `_conn()` з `row_factory=dict_row`, що повертає рядки як словники (зручно для друку у консолі).
- Кожен метод працює у блоці `with self._conn() as conn, conn.cursor() as cur;`; зміни фіксуються через `conn.commit()` після DML.
- Усі запити — параметризовані (`%s`, `execute(sql, params)`), що захищає від SQL-ін'єкцій.

```
class Model:
 def __init__(self, dsn: str):
 self._dsn = dsn

 def _conn(self):
 # row_factory=dict_row -> отримуємо dict-рядки
 return psycopg.connect(self._dsn, row_factory=dict_row)

 def ping(self) -> bool:
 try:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("SELECT 1;")
 cur.fetchone()
 return True
 except psycopg.Error:
 return False
```

Опис:

- `__init__(dsn: str)` — зберігає DSN підключення.
- `_conn()` — відкриває з'єднання до БД з `dict_row`.
- `ping()` — перевіряє доступність БД (`SELECT 1`), використовується при старті програми.

### 2.9.3. CRUD-функції за сутностями

#### A) Student

```

def student_list(self, limit=50, offset=0):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT student_id, last_name, first_name, email,
group_code
 FROM "Student" ORDER BY student_id
 LIMIT %s OFFSET %s;""",
 (limit, offset),
)
 return cur.fetchall()

def student_get(self, student_id:int):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT student_id, last_name, first_name, email,
group_code
 FROM "Student" WHERE student_id=%s;""",
 (student_id,),
)
 return cur.fetchone()

def student_create(self, last_name:str, first_name:str, email:str,
group_code:str) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """INSERT INTO "Student"(last_name, first_name, email,
group_code)
 VALUES (%s,%s,%s,%s) RETURNING student_id;""",
 (last_name, first_name, email, group_code),
)
 new_id = cur.fetchone()["student_id"]
 conn.commit()
 return new_id

def student_update(self, student_id:int, last_name:str, first_name:str,
email:str, group_code:str) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """UPDATE "Student"
 SET last_name=%s, first_name=%s, email=%s, group_code=%s
 WHERE student_id=%s;""",
 (last_name, first_name, email, group_code, student_id),
)
 conn.commit()
 return cur.rowcount

def student_delete(self, student_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""DELETE FROM "Student" WHERE student_id=%s;""",
(student_id,))
 conn.commit()
 return cur.rowcount

def count_enrollments_by_student(self, student_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""SELECT COUNT(*) AS cnt FROM "Enrollment" WHERE
student_id=%s;""", (student_id,))
 return cur.fetchone()["cnt"]

```

- `student_list(limit=50, offset=0)` — перелік студентів (пагінація, сортування за `student_id`).
- `student_get(student_id:int)` — отримати один запис за РК.
- `student_create(last_name, first_name, email, group_code) -> int` — вставка, повертає новий `student_id`.
- `student_update(student_id, last_name, first_name, email, group_code) -> int` — повне оновлення полів.
- `student_delete(student_id) -> int` — видалення студента за РК.
- `count_enrollments_by_student(student_id) -> int` — контроль **1:N**: рахує залежні рядки у `Enrollment` (використовується `Controller`-ом, щоб заборонити видалення батьківського запису при наявності підлеглих).

## B) Instructor

```

def instructor_list(self, limit=50, offset=0):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT instructor_id, last_name, first_name, email,
department
 FROM "Instructor" ORDER BY instructor_id
 LIMIT %s OFFSET %s;""",
 (limit, offset),
)
 return cur.fetchall()

def instructor_get(self, instructor_id:int):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT instructor_id, last_name, first_name, email,
department
 FROM "Instructor" WHERE instructor_id=%s;""",
 (instructor_id,),
)
 return cur.fetchone()

def instructor_create(self, last_name:str, first_name:str, email:str,
department:str) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """INSERT INTO "Instructor"(last_name, first_name, email,
department)
 VALUES (%s,%s,%s,%s) RETURNING instructor_id;""",
 (last_name, first_name, email, department),
)
 new_id = cur.fetchone()["instructor_id"]
 conn.commit()
 return new_id

def instructor_update(self, instructor_id:int, last_name:str,
first_name:str, email:str, department:str) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """UPDATE "Instructor"
 SET last_name=%s, first_name=%s, email=%s, department=%s
 WHERE instructor_id=%s;""",
 (last_name, first_name, email, department, instructor_id),
)
 conn.commit()
 return cur.rowcount

def instructor_delete(self, instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""DELETE FROM "Instructor" WHERE
instructor_id=%s;""", (instructor_id,))
 conn.commit()
 return cur.rowcount

def count_enrollments_by_instructor(self, instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""SELECT COUNT(*) AS cnt FROM "Enrollment" WHERE
instructor_id=%s;""", (instructor_id,))
 return cur.fetchone()["cnt"]

```

- `instructor_list(...)` / `instructor_get(...)` — перелік/отримання.
- `instructor_create(...)` -> `int` — вставка й повернення `instructor_id`.
- `instructor_update(...)` -> `int` — оновлення полів.
- `instructor_delete(...)` -> `int` — видалення за РК.
- `count_enrollments_by_instructor(instructor_id)` -> `int` — контроль залежностей у `Enrollment/Exam`.



## C) Course

```

def course_list(self, limit=50, offset=0):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT course_id, title, credits, semester
 FROM "Course" ORDER BY course_id
 LIMIT %s OFFSET %s;""",
 (limit, offset),
)
 return cur.fetchall()

def course_get(self, course_id:int):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT course_id, title, credits, semester
 FROM "Course" WHERE course_id=%s;""",
 (course_id,),
)
 return cur.fetchone()

def course_create(self, title:str, credits:int, semester:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """INSERT INTO "Course"(title, credits, semester)
 VALUES (%s,%s,%s) RETURNING course_id;""",
 (title, credits, semester),
)
 new_id = cur.fetchone()["course_id"]
 conn.commit()
 return new_id

def course_update(self, course_id:int, title:str, credits:int,
semester:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """UPDATE "Course"
 SET title=%s, credits=%s, semester=%s
 WHERE course_id=%s;""",
 (title, credits, semester, course_id),
)
 conn.commit()
 return cur.rowcount

def course_delete(self, course_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""DELETE FROM "Course" WHERE course_id=%s;""",
(course_id,))
 conn.commit()
 return cur.rowcount

def count_enrollments_by_course(self, course_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute("""SELECT COUNT(*) AS cnt FROM "Enrollment" WHERE
course_id=%s;""", (course_id,))
 return cur.fetchone()["cnt"]

```

- `course_list(...)`, `course_get(...)` — перелік/отримання.
- `course_create(title, credits, semester) -> int` — вставка курсу.
- `course_update(course_id, title, credits, semester) -> int` — оновлення.
- `course_delete(course_id) -> int` — видалення.
- `count_enrollments_by_course(course_id) -> int` — контроль залежностей (1:N).

#### D) Enrollment (PK: `course_id`, `student_id`, `instructor_id`)

```
def enrollment_list(self, limit=50, offset=0):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT en.course_id, c.title, en.student_id, s.last_name
AS s_last, s.first_name AS s_first,
 en.instructor_id, i.last_name AS i_last,
i.first_name AS i_first
FROM "Enrollment" en
JOIN "Course" c ON c.course_id=en.course_id
JOIN "Student" s ON s.student_id=en.student_id
JOIN "Instructor" i ON i.instructor_id=en.instructor_id
ORDER BY en.course_id, en.student_id, en.instructor_id
LIMIT %s OFFSET %s;""",
 (limit, offset),
)
 return cur.fetchall()

def enrollment_exists(self, course_id:int, student_id:int,
instructor_id:int) -> bool:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT 1 FROM "Enrollment"
WHERE course_id=%s AND student_id=%s AND
instructor_id=%s;""",
 (course_id, student_id, instructor_id),
)
 return cur.fetchone() is not None

def enrollment_create(self, course_id:int, student_id:int,
instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """INSERT INTO "Enrollment"(course_id, student_id,
instructor_id)
VALUES (%s,%s,%s) ON CONFLICT DO NOTHING;""",
 (course_id, student_id, instructor_id),
)
 conn.commit()
 return cur.rowcount # 1 якщо додано, 0 якщо існує

def enrollment_delete(self, course_id:int, student_id:int,
instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """DELETE FROM "Enrollment"
WHERE course_id=%s AND student_id=%s AND
instructor_id=%s;""",
 (course_id, student_id, instructor_id),
)
 conn.commit()
 return cur.rowcount
```

```

def count_exams_for_enrollment(self, course_id:int, student_id:int,
instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT COUNT(*) AS cnt FROM "Exam"
 WHERE course_id=%s AND student_id=%s AND
instructor_id=%s;""",
 (course_id, student_id, instructor_id),
)
 return cur.fetchone() ["cnt"]

```

- enrollment\_list(limit, offset) — перелік трійок з JOIN на назви курсу та ПІБ.
- enrollment\_exists(course\_id, student\_id, instructor\_id) -> bool — існування трійки (перевірка перед вставкою в Exam).
- enrollment\_create(course\_id, student\_id, instructor\_id) -> int — вставка трійки, ON CONFLICT DO NOTHING (унікальний РК).
- enrollment\_delete(course\_id, student\_id, instructor\_id) -> int — видалення трійки.
- count\_exams\_for\_enrollment(course\_id, student\_id, instructor\_id) -> int — чи є залежні Exam (Controller блокує видалення, якщо >0).

### E) Exam (PK: student\_id, course\_id, instructor\_id, attempt\_no)

```

def exam_list(self, limit=50, offset=0):
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT ex.student_id, s.last_name AS s_last, s.first_name
AS s_first,
 ex.course_id, c.title,
 ex.instructor_id, i.last_name AS i_last,
i.first_name AS i_first,
 ex.attempt_no, ex.document, ex.exam_date, ex.grade
FROM "Exam" ex
JOIN "Student" s ON s.student_id=ex.student_id
JOIN "Course" c ON c.course_id=ex.course_id
JOIN "Instructor" i ON i.instructor_id=ex.instructor_id
ORDER BY ex.exam_date DESC
LIMIT %s OFFSET %s;""",
 (limit, offset),
)
 return cur.fetchall()

def exam_create(self, student_id:int, course_id:int, instructor_id:int,
 attempt_no:int, document:str, exam_date:str, grade:int) -
> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """INSERT INTO "Exam"(student_id, course_id, instructor_id,
attempt_no, document, exam_date, grade)
VALUES (%s,%s,%s,%s,%s,%s,%s);""",
 (student_id, course_id, instructor_id, attempt_no, document,
exam_date, grade),
)
 conn.commit()
 return cur.rowcount

def exam_delete(self, student_id:int, course_id:int, instructor_id:int,
 attempt_no:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """DELETE FROM "Exam"
WHERE student_id=%s AND course_id=%s AND instructor_id=%s
AND attempt_no=%s;""",
 (student_id, course_id, instructor_id, attempt_no),
)
 conn.commit()
 return cur.rowcount

def exam_get_next_attempt(self, student_id:int, course_id:int,
instructor_id:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(
 """SELECT COALESCE(MAX(attempt_no),0)+1 AS next_no
FROM "Exam"
WHERE student_id=%s AND course_id=%s AND
instructor_id=%s;""",
 (student_id, course_id, instructor_id),
)
 return cur.fetchone()["next_no"]

def exam_update(self, student_id:int, course_id:int, instructor_id:int,
 attempt_no:int,
 document:str, exam_date:str, grade:int) -> int:
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(

```

```

 '''UPDATE "Exam"
 SET document=%s, exam_date=%s, grade=%s
 WHERE student_id=%s AND course_id=%s AND instructor_id=%s
AND attempt_no=%s;'''
 (document, exam_date, grade, student_id, course_id,
instructor_id, attempt_no),
)
 conn.commit()
 return cur.rowcount

```

- exam\_list(limit, offset) — перелік спроб із JOIN на студентів, курс і викладача.
- exam\_create(student\_id, course\_id, instructor\_id, attempt\_no, document, exam\_date, grade) -> int — вставка спроби (перед цим у Controller перевіряється наявність трійки у Enrollment).
- exam\_delete(student\_id, course\_id, instructor\_id, attempt\_no) -> int — видалення конкретної спроби.
- exam\_get\_next\_attempt(student\_id, course\_id, instructor\_id) -> int — обчислення наступного attempt\_no (MAX+1).
- exam\_update(student\_id, course\_id, instructor\_id, attempt\_no, document, exam\_date, grade) -> int — оновлення неключових полів спроби.

## 2.9.4. Автоматична генерація великих даних (тільки SQL)

```
def generate_students(self, n:int) -> int:
 sql = """
 INSERT INTO "Student"(last_name, first_name, email, group_code)
 SELECT
 'Last_' || gs::text,
 'First_' || gs::text,
 'student_' || gs::text || '@example.edu',
 'KP-' || (10 + (random()*20)::int)::text
 FROM generate_series(1, %s) AS gs;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (n,))
 conn.commit()
 return cur.rowcount

def generate_instructors(self, n:int) -> int:
 m = max(1, n // 100)
 sql = """
 INSERT INTO "Instructor"(last_name, first_name, email, department)
 SELECT
 'InstrL_' || gs::text,
 'InstrF_' || gs::text,
 'instructor_' || gs::text || '@univ.edu',
 'Dept_' || (1 + (random()*10)::int)::text
 FROM generate_series(1, %s) AS gs;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (m,))
 conn.commit()
 return cur.rowcount

def generate_courses(self) -> int:
 sql = """
 INSERT INTO "Course"(title, credits, semester)
 SELECT t,
 (1 + (random()*5)::int)::smallint,
 (1 + (random()*8)::int)::smallint
 FROM unnest(ARRAY[
 'Databases', 'Algorithms', 'Discrete Math', 'Networks',
 'OS', 'AI', 'ML', 'SE', 'Web'
]) AS t;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql)
 conn.commit()
 return cur.rowcount

def generate_enrollments(self, n:int) -> int:
 sql = """
 WITH s AS (SELECT student_id FROM "Student" ORDER BY random() LIMIT
 GREATEST(1, %s/2)),
 c AS (SELECT course_id FROM "Course" ORDER BY random() LIMIT
 10),
 i AS (SELECT instructor_id FROM "Instructor" ORDER BY random()
 LIMIT 20)
 INSERT INTO "Enrollment"(course_id, student_id, instructor_id)
 SELECT c.course_id, s.student_id, i.instructor_id
 FROM s CROSS JOIN c CROSS JOIN i
 ON CONFLICT DO NOTHING;
 """
 with self._conn() as conn, conn.cursor() as cur:
```

```

 cur.execute(sql, (n,))
 conn.commit()
 return cur.rowcount

def generate_exams(self, n:int) -> int:
 sql = """
 WITH e AS (
 SELECT course_id, student_id, instructor_id
 FROM "Enrollment"
 ORDER BY random()
 LIMIT %s
),
 attempts AS (
 SELECT e.*, gs AS attempt_no
 FROM e
 JOIN generate_series(1, (1 + (random()*3)::int)) AS gs ON TRUE
)
 INSERT INTO "Exam"(student_id, course_id, instructor_id, attempt_no,
document, exam_date, grade)
 SELECT
 student_id, course_id, instructor_id, attempt_no,
 'doc_' || md5(random()::text),
 (CURRENT_DATE - ((random()*1095)::int)::text::interval)::date,
 (random()*100)::int
 FROM attempts
 ON CONFLICT DO NOTHING;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (n,))
 conn.commit()
 return cur.rowcount

```

- generate\_students(n:int) -> int — масова вставка n студентів через generate\_series() і random().
- generate\_instructors(n:int) -> int — вставка приблизно max(1, n/100) викладачів.
- generate\_courses() -> int — вставка фіксованого пулу курсів через unnest(array[...]).
- generate\_enrollments(n:int) -> int — формування трійок: випадкові підмножини Student/Course/Instructor, декартів добуток і ON CONFLICT DO NOTHING для унікального РК.
- generate\_exams(n:int) -> int — вибір випадкових трійок з Enrollment, генерація 1–3 спроб на трійку, випадкові exam\_date (останні ~3 роки) та grade (0..100), ON CONFLICT DO NOTHING.

### 2.9.5. Складні пошуки (JOIN + фільтри/групування) — 3 методи

```
def search_1(self, group_code:str, grade_min:int, grade_max:int,
 course_title_pattern:str, date_from:str, date_to:str):
 sql = """
 SELECT s.student_id, s.last_name, s.first_name, c.title,
 ex.attempt_no, ex.exam_date, ex.grade
 FROM "Exam" ex
 JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
 JOIN "Student" s ON s.student_id = ex.student_id
 JOIN "Course" c ON c.course_id = ex.course_id
 WHERE s.group_code = %s
 AND ex.grade BETWEEN %s AND %s
 AND c.title ILIKE %s
 AND ex.exam_date BETWEEN %s AND %s
 ORDER BY s.last_name, s.first_name, ex.exam_date DESC;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (group_code, grade_min, grade_max,
 course_title_pattern, date_from, date_to))
 return cur.fetchall()

def search_2(self, date_from:str, date_to:str, min_attempts:int):
 sql = """
 SELECT c.title,
 i.last_name || ' ' || i.first_name AS instructor,
 COUNT(*) AS attempts,
 ROUND(AVG(ex.grade),1) AS avg_grade
 FROM "Exam" ex
 JOIN "Enrollment" en USING (student_id, course_id, instructor_id)
 JOIN "Course" c ON c.course_id = ex.course_id
 JOIN "Instructor" i ON i.instructor_id = ex.instructor_id
 WHERE ex.exam_date BETWEEN %s AND %s
 GROUP BY c.title, i.last_name, i.first_name
 HAVING COUNT(*) >= %s
 ORDER BY avg_grade DESC;
 """
 with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (date_from, date_to, min_attempts))
 return cur.fetchall()

def search_3(self, semester:int, min_credits:int, fail_threshold:int):
 # (враховуємо, що студент міг перескласти, тому беремо останню
 спробу)
 sql = """
 WITH last_attempt AS (
 SELECT
 ex.student_id, ex.course_id, ex.instructor_id,
 ex.attempt_no, ex.document, ex.exam_date, ex.grade,
 ROW_NUMBER() OVER (
 PARTITION BY ex.student_id, ex.course_id,
ex.instructor_id
 ORDER BY ex.exam_date DESC, ex.attempt_no DESC
) AS rn
 FROM "Exam" ex
)
 SELECT
 s.student_id, s.last_name, s.first_name,
 c.title, c.semester, c.credits,
 la.grade AS last_grade,
 la.exam_date AS last_exam_date
 FROM last_attempt la
 JOIN "Enrollment" en
```



```

ON en.student_id = la.student_id
AND en.course_id = la.course_id
AND en.instructor_id = la.instructor_id
JOIN "Student" s ON s.student_id = la.student_id
JOIN "Course" c ON c.course_id = la.course_id
WHERE la.rn = 1
AND c.semester = %s
AND c.credits >= %s
AND la.grade < %s
ORDER BY last_grade ASC, last_exam_date DESC;
"""
with self._conn() as conn, conn.cursor() as cur:
 cur.execute(sql, (semester, min_credits, fail_threshold))
 return cur.fetchall()

```

- search\_1(group\_code, grade\_min, grade\_max, course\_title\_pattern, date\_from, date\_to)  
Повертає спроби Student–Course із фільтрами: група, діапазон оцінок (BETWEEN), шаблон назви курсу (LIKE), діапазон дат (BETWEEN). Сортує за ПІБ і датою.
- search\_2(date\_from, date\_to, min\_attempts)  
Агрегація Exam за курсом і викладачем за періодом дат: COUNT(\*) AS attempts, AVG(grade) AS avg\_grade. Фільтрує групи з достатньою кількістю спроб (HAVING COUNT(\*) >= ?), сортує за середнім балом.
- search\_3(semester, min\_credits, fail\_threshold)  
Враховує перескладання: спочатку CTE last\_attempt (вікно ROW\_NUMBER() OVER (...) за останніми exam\_date і attempt\_no), потім відбір лише тих, у кого останній бал < порогу. Додає до результату last\_grade, last\_exam\_date, а також атрибути курсу (semester, credits).

## 2.9.6. Помилки БД та відповідальність рівнів

- **Model** виконує запити і, за замовчуванням, пробрасує винятки psycopg.Error нагору.
- **Controller** перехоплює їх та виводить користувачу повідомлення (наприклад, ForeignKeyViolation, UniqueViolation).
- Контроль 1:N (блокування видалення батьківського запису при наявних дочірніх) реалізовано до виконання SQL за допомогою методів підрахунку залежностей (count\_\*).

### 3. Висновок:

У роботі реалізовано консольний застосунок для взаємодії з СУБД PostgreSQL відповідно до схеми БД у лабораторній роботі №1 та вимог РГР. Архітектура побудована за шаблоном MVC: модуль Model інкапсулює доступ до даних і виконує лише параметризовані SQL-запити через `psycopg3`; Controller відповідає за бізнес-логіку, перевірки та перехоплення помилок БД; View — за зручну взаємодію в консолі.

Забезпечено повний CRUD для сутностей Student, Instructor, Course, а також для Exam (включно з редагуванням неключових полів); для Enrollment (асоціативна трійка РК) передбачено додавання/видалення. Реалізовано контроль зв'язків 1:N: видалення батьківських рядків блокується за наявності підлеглих записів у Enrollment/Exam.

Масове наповнення БД виконано виключно засобами SQL (`generate_series()`, `random()`, `unnest()`), з параметром N, який задає користувач. Генерація відбувається в коректному порядку (спочатку батьківські таблиці, далі підлеглі), із застосуванням `ON CONFLICT DO NOTHING` для уникнення дублювань РК, що гарантує узгодженість із зовнішніми ключами. Для перевірки швидкодії підготовлено три пошукові запити з JOIN кількох сутностей, фільтрами для рядків/чисел/дат, групуванням та виміром часу виконання (мс).

Таким чином: розроблено працездатний застосунок, який повністю відповідає вимогам (консольний інтерфейс, MVC, лише SQL, обробка помилок без трейсбеків, масова генерація даних у БД, складні пошуки з виміром часу).